

Migração para o servidor AWS — manual de execução

Aplicação	WeeFly Concierge + Price Checker (Next.js 14)
De	<code>weefly.duckdns.org</code>
Para	<code>concierge.weefly.africa</code> — Elastic IP <code>52.30.78.0</code>
Servidor	EC2 <code>t3.medium</code> (2 vCPU, 4 GB), Ubuntu 24.04, fornecido pela equipa do Sarin Ram
Data	25 de Setembro de 2026 — revisto depois do reconhecimento e da resposta do Sarin sobre o IAM
Para seguir por	Fábio Rocha

Este documento é para ser seguido de cima a baixo, numa sessão. Cada parte acaba num estado verificável — se a verificação falhar, não se avança.

Não escrevas segredos neste documento. As chaves reais vêm do servidor antigo e do gestor de palavras-passe; este ficheiro circula por email.

Antes de começar

O que está confirmado

O Sarin respondeu às perguntas e o `recon.sh` correu no servidor a 25 de Setembro. Onde os dois discordam, vale o servidor.

Pergunta	Resposta
Utilizador SSH	<code>ubuntu</code> . O Sarin disse <code>ec2-user</code> , mas esse é recusado — a imagem é Ubuntu 24.04, não Amazon Linux.
Sistema	Ubuntu 24.04.4 LTS, glibc 2.39, disco de 29 GB ext4 com 25 GB livres, sem IPv6.
Node / PM2 / NGINX	Node 20.20, npm 10.8, PM2 7.0.4, NGINX 1.24, git 2.43 — tudo instalado.
Instância	<code>t3.medium</code> , 2 vCPU, 3,7 GB de RAM, ~3,1 GB livres, sem swap . Swap autorizado.
Permissões	<code>sudo</code> sem password. <code>/var/www/concierge</code> já existe, vazia, e pertence ao <code>ubuntu</code> .
Conta AWS	É do cliente. Utilizador IAM <code>weefly-concierge-team</code> entregue a 25 de Setembro, só com acesso à consola, limitado a esta instância. A password está no gestor de palavras-passe, não aqui.
Porta 3000	Fechada ao público.
HTTPS de saída	Aberto — Supabase, Resend, npm.
DNS de <code>concierge.weefly.africa</code>	Criado pela equipa técnica deles. São eles o contacto para registos futuros.
Certificado	Certbot 2.9, <code>certbot.timer</code> activo, válido até 30 de Novembro de 2026. O Sarin diz que foi instalado com <code>dnf</code> , mas o <code>dnf</code> não existe em Ubuntu: é o pacote <code>apt certbot 2.9.0-1</code> , confirmado no servidor. A renovação funciona na mesma.
PM2	Um único daemon, do <code>ubuntu</code> , sem processos. Nenhum <code>pm2 startup</code> registado ainda.
O que já corre	Só o NGINX (80, 443) e o sshd (22). O NGINX serve <code>/var/www/html</code> , ainda sem <code>proxy_pass</code> .
SSH	Só chave; <code>PasswordAuthentication no</code> . Porta 22 aberta a <code>0.0.0.0/0</code> .
Agente SSM	Instalado (snap) e activo. Instance profile <code>AmazonSSMManagedInstanceCore-Concierge</code> confirmado no servidor a 25 de Setembro; o agente autentica-se e está ligado ao Session Manager.
Instância	<code>i-0152d6578ff6a5801</code> , região <code>eu-west-1</code> (Irlanda), confirmadas pelos metadados da instância.
Snapshots EBS	Não configuradas. Ficam por nossa conta.

Acessos AWS

O quê	Estado	Próximo passo
Utilizador IAM	Recebido a 25 de Setembro	Primeiro login, trocar a password e activar MFA — ver abaixo.
Instance profile	Confirmado no servidor	Nada. O log do agente mostra sessões já abertas com weefly-concierge-team , às 13:12 e 13:23 UTC de 25 de Setembro — a equipa do Sarin a testar.
Snapshots EBS	Permissão pedida e concedida	Confirmar com uma snapshot real antes da instalação.
Fechar a porta 22	Do nosso lado	O lado técnico do SSM está provado. Falta uma sessão aberta por nós, no browser, com a password já trocada. Depois disso, avisar o Sarin. Recusaram o whitelisting porque o IPv4 é dinâmico.

O bloqueio deixou de estar do lado do Sarin: a 22 continua aberta a **0.0.0.0/0** até nós dizermos que o SSM funciona.

Verificar o acesso AWS — antes de tudo o resto

A password chegou por WhatsApp, junto com o URL e o utilizador. É o mesmo problema da chave SSH, com uma diferença: esta dá acesso à consola da conta do cliente.

1. **Primeiro login e troca de password.** O URL de consola e o utilizador estão no gestor de palavras-passe. Se a AWS não pedir a troca logo no login, troca-a em *Security credentials*. Se a troca for recusada por falta de permissão, pede ao Sarin que acrescente **iam:ChangePassword** para o próprio utilizador.
2. **MFA** no mesmo ecrã. Se for recusado, pede também esta permissão. Um utilizador de consola sem MFA, com uma password que passou por uma conversa, não pode ficar assim.
3. **Região.** Escolhe no canto superior direito **Europe (Ireland) eu-west-1**. A instância é a **i-0152d6578ff6a5801**. Uma região errada mostra uma consola vazia, que parece um problema de permissões e não é.
4. **Session Manager.** O lado do servidor já está confirmado: o instance profile está ligado e o agente está registado. Em Systems Manager → Fleet Manager, a instância aparece como *Online*. Depois, EC2 → a instância → *Connect* → *Session Manager* → *Connect*. Se a instância não aparecer, o agente arrancou antes de o profile existir. Reinicia-o pela 22, que ainda está aberta:

```
sudo snap restart amazon-ssm-agent
```

5. **Na sessão, muda logo de utilizador.** O Session Manager entra como **ssm-user**, não como **ubuntu**. O PM2, a pasta da aplicação e o **pm2 startup** são todos do **ubuntu** (Parte 5):

```
sudo -iu ubuntu
whoami      # ubuntu
```

6. **Snapshot de teste.** EC2 → Volumes → o volume da instância → *Create snapshot*, com a descrição **antes-da-instalacao**. Serve para confirmar a permissão e fica como ponto de partida limpo.

7. **Pedir o fecho da 22.** Com os pontos 4 a 6 confirmados, avisa o Sarin. Antes disso, confirma numa sessão SSM que consegues fazer tudo o que vais precisar: `git pull`, `nano` do `.env.production` e `pm2`. Depois de fechada, não há `scp` nem `ssh`. O `recon.sh` corre colando o conteúdo na sessão.

Estado verificável: uma sessão do Session Manager aberta, `whoami` a dar `ubuntu`, e uma snapshot em estado *Completed*.

Até a porta 22 fechar

A porta 22 está aberta à internet. Não é motivo para esperar, mas a Parte 11 passa a vir **no mesmo dia**, e não "depois de estar a funcionar": trocar a chave que viajou por mensagem e confirmar que a autenticação por password está desligada.

Parte 1 · As chaves e o primeiro acesso

Tirar as chaves da pasta do repositório

Feito a 25 de Setembro. As chaves chegaram à raiz do repositório e já estão em `~/ssh`. Não foram para o git — o `.gitignore` apanha `*.pem` e `*.ppk` — mas a pasta do repositório é o sítio errado: basta um `git add -f`, um zip para mandar a alguém, ou uma sincronização de backup.

```
mkdir -p ~/.ssh
mv concierge-weefly-key.pem concierge-weefly-key.ppk ~/.ssh/
```

O OpenSSH do Windows recusa a chave se as permissões forem largas, com `UNPROTECTED PRIVATE KEY FILE`. Corrige antes de tentar ligar:

```
icacls "$USERPROFILE\.ssh\concierge-weefly-key.pem" /inheritance:r /grant:r "$USERNAME:R"
```

Entrar

```
ssh -i ~/.ssh/concierge-weefly-key.pem ubuntu@52.30.78.0
```

`ec2-user` e `admin` dão `Permission denied (publickey)`; `root` pede que se use `ubuntu`.

Com o SSM pronto, a entrada passa a ser pela consola: EC2 → a instância → *Connect* → *Session Manager*. Não precisa de chave nem da porta 22. Logo a seguir, `sudo -iu ubuntu`.

O utilizador IAM é só de consola, sem access keys, por isso o `aws ssm start-session` da linha de comandos não funciona. O browser chega.

Reconhecimento

Feito a 25 de Setembro — os resultados estão na tabela do início. Para voltar a correr, da tua máquina, sem copiar nada para o servidor:

```
ssh -i ~/.ssh/concierge-weefly-key.pem ubuntu@52.30.78.0 'sh -s' < deploy/recon.sh
```

Só lê. O que interessa para o resto do documento:

- Node 20 sobre glibc 2.39 — a Next 14 corre sem instalar nada.
- O bloco `server` do Certbot está em `/etc/nginx/sites-enabled/concierge.weefly.africa`. É esse o ficheiro da Parte 6.
- Não há swap. A Parte 3 é para fazer.

Estado verificável: tens sessão, sabes a versão do sistema, sabes onde vive a configuração do NGINX.

Parte 2 · O código no servidor

A chave de deploy do GitHub

O repositório é privado. No servidor, gera uma chave dedicada e só de leitura — não uses o teu token pessoal, que dá acesso a tudo o que é teu:

```
ssh-keygen -t ed25519 -f ~/.ssh/github-deploy -N "" -C "ec2-concierge"
cat ~/.ssh/github-deploy.pub
```

Copia o resultado e vai a **github.com/FabioRocha-pt/Weefly-Backend** → **Settings** → **Deploy keys** → **Add deploy key**. Cola, dá-lhe um nome (**EC2 concierge**) e **deixa a caixa de escrita desmarcada**.

Depois, ensina o SSH a usá-la:

```
cat >> ~/.ssh/config <<'EOF'
Host github.com
  IdentityFile ~/.ssh/github-deploy
  IdentitiesOnly yes
EOF
chmod 600 ~/.ssh/config
ssh -T git@github.com      # deve responder "successfully authenticated"
```

Clonar

```
sudo mkdir -p /var/www/concierge
sudo chown -R $USER:$USER /var/www/concierge
git clone git@github.com:FabioRocha-pt/Weefly-Backend.git /var/www/concierge
cd /var/www/concierge && git log --oneline -1
```

Estado verificável: o `git log` mostra o último commit do `main`.

Parte 3 · Memória — fazer isto antes do primeiro build

O `next build` chega a pedir 1,5 GB. Numa instância pequena é morto pelo kernel **sem mensagem nenhuma**: o build pára e não há erro para ler. É a avaria que mais tempo faz perder porque não se parece com falta de memória.

A instância tem 4 GB, com cerca de 3,1 livres. Chega para o build. Mesmo assim, cria 2 GB de swap: o Sarin autorizou, custa um minuto, e é o que separa um build lento de um build morto no dia em que o NGINX, o PM2 e um `npm ci` coincidirem.

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
free -h                # a linha Swap deixou de estar a zero
```

A alternativa, se o disco for pequeno, é construir noutra máquina e enviar a pasta `.next` já construída. Dá mais trabalho a cada deploy.

Parte 4 · As variáveis de ambiente

O ficheiro é `/var/www/concierge/.env.production`, na mesma pasta do `package.json`. O `next start` lê-o sozinho no arranque — o PM2 não precisa de `env_file`.

Os valores vêm do servidor antigo. Entra no `weefly.duckdns.org`, faz `cat` ao `.env.production` de lá, e traz os valores um a um. Não os copies do `.env.local` da tua máquina: esse tem `NEXT_PUBLIC_SITE_URL=http://localhost:3000`, e esse valor errado manda cada cliente para um sítio que não existe — sem erro nenhum, porque os emails saem à mesma.

```
nano /var/www/concierge/.env.production
```

Obrigatórias — sem estas a aplicação não serve para nada

```
NEXT_PUBLIC_SUPABASE_URL=  
NEXT_PUBLIC_SUPABASE_ANON_KEY=  
SUPABASE_SERVICE_ROLE_KEY=  
  
NEXT_PUBLIC_SITE_URL=https://concierge.weefly.africa
```

O `NEXT_PUBLIC_SITE_URL` é o único valor que muda de conteúdo em relação ao servidor antigo. É ele que forma cada link de cliente e cada endereço dentro dos emails.

Email — sem estas o formulário responde bem e nada chega a ninguém

```
RESEND_API_KEY=  
CONCIERGE_FROM_EMAIL="WeeFly Concierge <concierge@weefly.africa>"  
CONCIERGE_TEAM_EMAIL=info@weefly.africa,info@weefly.cv  
RESEND_WEBHOOK_SECRET=
```

Operação

```
CONCIERGE_DIAGNOSE_TOKEN=  
PC_CRON_TOKEN=  
BO_ALLOWED_EMAILS=
```

O `CONCIERGE_DIAGNOSE_TOKEN` é uma cadeia aleatória à tua escolha — é o que destranca a rota de diagnóstico da Parte 9. Sem ela, a rota responde 404 em produção.

O `PC_CRON_TOKEN` é outra cadeia aleatória, e é obrigatória para a Parte 8. Gera as duas assim:

```
openssl rand -hex 24
```

O `BO_ALLOWED_EMAILS` é apenas uma rede de segurança: a lista real de acessos ao back-office vive na tabela `bo_allowlist` da base de dados, e a variável é ignorada quando a tabela responde. Traz o valor do servidor antigo na mesma.

Opcionais — trazer do servidor antigo se lá estiverem

```
ANTHROPIC_API_KEY=  
CONCIERGE_NLP_MODEL=  
AMADEUS_CLIENT_ID=  
AMADEUS_CLIENT_SECRET=  
AMADEUS_API_BASE=  
AMADEUS_CURRENCY=  
WEEPAY_API_URL=  
WEEPAY_API_KEY=  
WEEPAY_WEBHOOK_SECRET=  
WEEPAY_DEFAULT_COUNTRY=  
WEEPAY_DEFAULT_METHOD=  
WHATSAPP_PHONE_NUMBER_ID=  
WHATSAPP_ACCESS_TOKEN=  
WHATSAPP_TEAM_NUMBER=  
WHATSAPP_API_VERSION=
```

As `BO_PASSWORD_*` que existem no teu `.env.local` **não vão para o servidor**. São usadas só pelo `scripts/seed-bo-users.mjs`, que corre da tua máquina.

Fechar o ficheiro

```
chmod 600 /var/www/concierge/.env.production
```

Tem lá dentro a chave de serviço do Supabase, que passa por cima de todas as políticas de RLS. Qualquer pessoa com esse valor lê e escreve tudo.

Estado verificável: `ls -l .env.production` mostra `-rw-----`.

Parte 5 · Construir e arrancar

```
cd /var/www/concierge
npm ci                # ci, não install: respeita o package-lock
npm run build
```

Se o build parar sem erro, volta à Parte 3.

O PM2 corre como `ubuntu`, nunca com `sudo`. A lista de processos do PM2 é por utilizador: um `sudo pm2 ...` — até um `sudo pm2 list` — arranca um segundo daemon em `/root/.pm2`, e o `pm2 startup` acaba registado no utilizador errado. Nenhum comando `pm2` leva `sudo` à frente, excepto o que o `pm2 startup` imprime. Numa sessão do Session Manager, isto também quer dizer começar com `sudo -iu ubuntu`: o `ssm-user` tem o seu próprio PM2, vazio.

```
mkdir -p logs
pm2 start ecosystem.config.js
pm2 save
pm2 startup          # imprime um comando com sudo — confirma que tem
                    # "-u ubuntu --hp /home/ubuntu", copia-o e corre-o
pm2 logs weefly-concierge --lines 40
```

O `ecosystem.config.js` já está no repositório e já está escrito para este servidor: uma instância, `127.0.0.1:3000`, `TZ=UTC`, reinício com espera crescente e tecto de memória a 700 MB.

O `pm2 startup` é o que faz a aplicação sobreviver a um reboot da máquina. Saltar esse passo só se nota no dia em que a AWS reinicia a instância.

Estado verificável:

```
curl -I http://127.0.0.1:3000    # HTTP/1.1 200 OK
pm2 status                      # weefly-concierge · online · 0 restarts
```

Parte 6 · O NGINX — o passo que parte em silêncio

Lê isto antes de mexer. O cliente carrega o comprovativo de pagamento no ecrã dele — JPG, PNG ou PDF, até 8 MB. O `client_max_body_size` do NGINX está por omissão em **1MB**. Uma fotografia de um comprovativo tirada com um telemóvel moderno tem 3 a 6 MB.

O que acontece sem esta alteração: o NGINX corta o pedido com **413 antes de ele chegar à aplicação**. O cliente vê uma falha de upload que nenhum registo da aplicação explica, porque o pedido nunca lá chegou.

Abre `/etc/nginx/sites-enabled/concierge.weefly.africa`. Hoje o bloco do 443 serve `/var/www/html` com `try_files`; essa `location /` sai toda. Garante que o bloco `server` do 443 fica assim:

```
server {
    listen 443 ssl http2;
    server_name concierge.weefly.africa;

    # — ssl_certificate ... o que o Certbot já configurou, não mexer —

    # O comprovativo do cliente: 8 MB mais a folga do multipart.
    client_max_body_size 12M;
    client_body_timeout 120s;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;

        proxy_set_header Host                $host;
        proxy_set_header X-Real-IP            $remote_addr;
        proxy_set_header X-Forwarded-For      $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto    $scheme;

        proxy_read_timeout 90s;
        proxy_buffering off;
    }

    location /_next/static/ {
        proxy_pass http://127.0.0.1:3000;
        proxy_cache_valid 200 365d;
        add_header Cache-Control "public, max-age=31536000, immutable";
    }
}
```

O `X-Forwarded-For` não é decorativo: o ecrã de consentimento promete guardar o IP de quem o deu, e a aplicação lê-o desse cabeçalho. Sem ele, todos os consentimentos ficam registados como vindos de `127.0.0.1`.

```
sudo nginx -t && sudo systemctl reload nginx
```

Não é preciso configuração de WebSocket. As actualizações em tempo real do back-office vão do browser directamente para o Supabase, não passam por aqui.

Estado verificável: `curl -I https://concierge.weefly.africa` responde 200.

Parte 7 · O que muda fora do servidor

Três sítios que não estão na máquina e que ficam a apontar para o endereço antigo se ninguém lá for.

Onde	O que mudar
Painel do Resend	O webhook passa a <code>https://concierge.weefly.africa/api/webhooks/resend</code>
Supabase → Authentication → URL Configuration	Acrescentar <code>https://concierge.weefly.africa</code> ao Site URL e às Redirect URLs
Painel da WeePay, se já estiver ligado	O webhook passa a <code>https://concierge.weefly.africa/api/weepay/webhook</code>

O Supabase é o que parte o login do back-office: a aplicação constrói o retorno como `{origin}/auth/callback`, e o Supabase recusa redirecionar para um endereço que não esteja na lista.

Parte 8 · O cron que não existe neste servidor

No Vercel havia uma entrada em `vercel.json` a chamar `/api/pc/expire` de hora a hora. **Neste servidor não há nada equivalente, e é preciso criar.**

A rota fecha os pagamentos cujo prazo já passou. A expiração também acontece preguiçosamente sempre que alguém abre a página do cliente ou a ficha do back-office — o cron existe para o caso em que ninguém abre nem uma nem outra. Sem ele, um cliente que só volta ao link daqui a uma semana vê o ecrã de pagamento como se o preço ainda valesse.

É um timer de `systemd` e não uma linha de `crontab`: fica registado com estado e histórico (`systemctl status`), recupera uma hora perdida depois de um reboot, e o token não fica numa linha de `crontab`. Os dois ficheiros já estão no repositório, em `deploy/systemd/`. O token vai para um ficheiro só de root, não para dentro da unidade:

```
sudo mkdir -p /etc/weefly
echo "PC_CRON_TOKEN=O_MESMO_VALOR_DO_ENV_PRODUCTION" | sudo tee /etc/weefly/pc-cron.env > /dev/null
sudo chmod 600 /etc/weefly/pc-cron.env

sudo cp /var/www/concierge/deploy/systemd/weefly-pc-expire.* /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now weefly-pc-expire.timer
```

Estado verificável:

```
sudo systemctl start weefly-pc-expire.service    # corre-o agora, à mão
systemctl status weefly-pc-expire.service        # "status=0/SUCCESS"
systemctl list-timers | grep weefly              # a próxima execução está marcada
```

Se o estado mostrar `curl: (22) ... 404`, o `PC_CRON_TOKEN` não está no `.env.production` ou a aplicação não foi reiniciada depois de o acrescentar. Se mostrar `401`, o valor em `/etc/weefly/pc-cron.env` é diferente do da aplicação.

Parte 9 · Os links antigos do duckdns

Há links `weefly.duckdns.org` já entregues a clientes — em emails enviados e em mensagens de WhatsApp. Se desligares a máquina antiga, esses links morrem.

O `duckdns` é teu, portanto tens uma saída limpa: aponta `weefly.duckdns.org` para `52.30.78.0` no painel do DuckDNS e deixa o servidor novo responder por ambos os nomes, redireccionando.

Depois de o DNS propagar:

```
sudo certbot --nginx -d weefly.duckdns.org
```

E um bloco que reencaminha tudo, preservando o caminho:

```
server {  
    listen 443 ssl http2;  
    server_name weefly.duckdns.org;  
  
    # — ssl_certificate ... o que o Certbot acabou de configurar —  
  
    return 301 https://concierge.weefly.africa$request_uri;  
}
```

Um link antigo como `/pc/8IdIXN__M40YN1YoHjYmtTngIJJaQj0rh` passa a chegar ao sítio certo, e passas a ter um servidor só.

Confirma que não há nada em falta antes de desligar a máquina antiga: no código não existe nenhum `duckdns` escrito à mão — está verificado. O domínio entra todo pelo `NEXT_PUBLIC_SITE_URL`.

Parte 10 · Antes de dizer que está no ar

Por esta ordem. Cada um apanha uma avaria diferente.

- **As migrações estão aplicadas.** São 19, até à `0019_sprint3_results.sql`. Confirma no SQL Editor do Supabase. Sem elas a aplicação arranca e só falha quando alguém a usa.
 - `https://conciierge.weefly.africa/api/conciierge/diagnose?token=...` responde e não traz `blockers`.
 - **Submeter um pedido real em /pc**. Tem de chegar o email de confirmação ao cliente e o alerta à equipa.
 - **Carregar um comprovativo de 5 MB.** É este o teste que apanha o `client_max_body_size`. Um ficheiro de 200 KB passa com a configuração errada e dá falsa confiança.
 - **Abrir esse comprovativo no back-office.**
 - **Emitir um caso de teste** e confirmar que o PDF aparece no link do cliente.
 - **Na aba Comunicações do caso**, os avisos passam de `enviado` a `entregue`. Se ficarem em `enviado`, falta o webhook do Resend da Parte 7.
 - **Entrar no back-office** com uma conta real. Se o login rodar e voltar ao início, falta o Supabase da Parte 7.
 - **Abrir um link antigo do duckdns** e confirmar que aterra no domínio novo.
-

Parte 11 · Segurança — no mesmo dia

Enquanto a porta 22 estiver aberta à internet, isto não fica para depois.

- **Sem autenticação por password** — confirmado a 25 de Setembro: `passwordauthentication no`, `permitrootlogin without-password`.
- **Substituir a chave SSH**. A chave que recebeste viajou por mensagem: está no telemóvel de quem a enviou, no histórico da conversa e em todos os backups dessa conversa. Deixou de ser privada em qualquer sentido útil.

```
ssh-keygen -t ed25519 -f ~/.ssh/weefly-concierge -C "fabio@bonako"
ssh-copy-id -i ~/.ssh/weefly-concierge.pub ubuntu@52.30.78.0
```

Testa a chave nova numa segunda sessão **antes** de fechar a primeira. Só depois de entrares com ela é que pedes ao Sarin para remover a original. Se a 22 já estiver fechada, este passo perde a urgência: a chave deixa de abrir alguma coisa. Remove-se a original na mesma, pelo Session Manager, em `~/.ssh/authorized_keys`.

- **Portas abertas**: 443, e 80 para o redireccionamento e a renovação do certificado. A 22 fecha assim que o SSM Session Manager funcionar. A 3000 já está fechada, confirmado pelo Sarin.
- **Renovação do certificado**. O `certbot.timer` está activo e o certificado é válido até 30 de Novembro. Falta provar que a renovação passa **depois** de a Parte 6 mexer no ficheiro do NGINX:

```
sudo certbot renew --dry-run      # tem de acabar sem erros
```

Um certificado que expira ao domingo derruba o site e os links de todos os clientes.

- **Snapshot do EBS** depois de tudo configurado, como a de teste de "Verificar o acesso AWS", com a descrição `app-instalada`. A máquina não guarda dados — está tudo no Supabase — mas guarda a configuração, e é essa que custa uma tarde a refazer.
-

Deploys a partir daqui

```
cd /var/www/concierge  
git pull  
npm ci  
npm run build  
pm2 reload weefly-concierge
```

O `.env.production` não é tocado por nenhum destes comandos. Se acrescentares uma variável, tens de reiniciar: as variáveis são lidas no arranque.

Se correr mal

A máquina antiga fica de pé até tudo estar verificado. Voltar atrás é apontar o DNS de `concierge.weefly.africa` para o IP antigo e repor o `NEXT_PUBLIC_SITE_URL` que lá estava. Nenhum dado se perde no processo, porque nenhum dado vive nestas máquinas.

Sintoma	Causa provável
<code>Permission denied (publickey)</code>	Utilizador diferente de <code>ubuntu</code> , ou chave errada
Ligação em timeout	Security group, não a chave — a 22 pode já ter sido fechada a favor do SSM
Consola da AWS vazia, sem a instância	Região errada no canto superior direito
Instância não aparece no Fleet Manager	Agente arrancou antes do instance profile — <code>sudo snap restart amazon-ssm-agent</code>
<code>pm2 list</code> vazio numa sessão SSM	Estás como <code>ssm-user</code> — <code>sudo -iu ubuntu</code>
App não volta depois de reboot, mas <code>sudo pm2 list</code> mostra-a	PM2 registado como root — Parte 5
<code>UNPROTECTED PRIVATE KEY FILE</code>	Permissões do <code>.pem</code> no Windows — Parte 1
Build pára sem erro	Falta de memória — Parte 3
Upload de comprovativo falha, sem registo na aplicação	<code>client_max_body_size</code> — Parte 6
Formulário aceita e nenhum email sai	<code>RESEND_API_KEY</code> em falta; é deliberado que não dê erro
Links dos emails apontam para <code>localhost</code>	<code>NEXT_PUBLIC_SITE_URL</code> copiado do <code>.env.local</code>
Avisos ficam em <code>enviado</code> , nunca <code>entregue</code>	Webhook do Resend — Parte 7
Login do back-office roda e volta ao início	Redirect URLs no Supabase — Parte 7
<code>/api/concierge/diagnose</code> responde 404	<code>CONCIERGE_DIAGNOSE_TOKEN</code> em falta, ou aplicação não reiniciada
<code>/api/pc/expire</code> responde 404	<code>PC_CRON_TOKEN</code> em falta, ou aplicação não reiniciada
Consentimentos registados como <code>127.0.0.1</code>	<code>X-Forwarded-For</code> em falta no NGINX — Parte 6
Site cai depois de um reboot da AWS	Faltou o <code>pm2 startup</code> — Parte 5